

Módulo 7 – Capítulo 09 – Sección 01

Arquitectura de despliegue: stateless vs stateful, sincrónico vs asincrónico

Llevar un agente de un prototipo funcional a producción exige decisiones arquitectónicas que van mucho más allá del código del agente en sí. El prototipo puede ejecutarse en el laptop del desarrollador porque opera sobre un solo usuario, un solo hilo de ejecución, y con la tolerancia de que si el proceso falla simplemente se reinicia manualmente. Un sistema de producción exige, en cambio, garantías de comportamiento bajo carga concurrente, recuperación automática ante fallos de instancias, latencia predecible para usuarios simultáneos, y costos controlables que escalen de forma proporcional a la demanda. Ninguna de estas propiedades es gratuita: cada una requiere decisiones arquitectónicas específicas tomadas antes del despliegue, no después del primer incidente de producción.

El primer eje de diseño es el **modelo de estado**. Un agente **stateless** no mantiene ningún estado en la memoria de la instancia entre requests: todo el estado del agente —el historial de la conversación, el contexto acumulado, la configuración de la sesión— se recupera de un almacenamiento externo (Redis para estado de sesión de baja latencia, PostgreSQL para historial persistente a largo plazo) al inicio de cada request y se persiste al final. La ventaja es fundamental para producción: cualquier instancia del agente puede procesar cualquier request de cualquier usuario, lo que permite el escalado horizontal mediante la simple adición de instancias sin complejidad adicional de routing. Si una instancia falla durante el escalado de una nueva versión, el siguiente request del usuario llega a otra instancia que recupera el estado del almacenamiento externo y continúa sin que el usuario note la interrupción.

Un agente **stateful** mantiene el estado en la memoria de la instancia: la conversación en progreso, el contexto cargado, los resultados de herramientas ya invocadas. Esto simplifica el código del agente porque no requiere serialización y deserialización del estado en cada request, pero introduce complejidad operativa significativa: el load balancer necesita sticky routing (el mismo usuario siempre llega a la misma instancia), lo que desequilibra la carga y complica el rolling deployment. Si la instancia que mantiene el estado de un usuario falla, esa sesión se pierde irrecuperablemente. Para la mayoría de los sistemas de producción, la

arquitectura stateless con persistencia externa es la opción correcta, independientemente de la complejidad adicional de implementación.

El segundo eje es el **modelo de ejecución temporal**. El modo **síncrono con streaming** es apropiado para tareas cuya latencia total es inferior a 30-60 segundos: el servidor mantiene la conexión abierta y envía los pasos del agente en tiempo real mediante Server-Sent Events (SSE) o WebSocket —cada tool call, cada observación, cada fragmento del razonamiento visible— mientras el agente ejecuta, de forma que el usuario percibe actividad continua aunque el resultado final tarde 30 segundos. El modo **asíncrono con job queue** es necesario para tareas de mayor duración: el request del cliente encola la tarea en una cola de mensajes (Celery + Redis, SQS, Bull.js) y recibe inmediatamente un `job_id` de confirmación; un worker independiente ejecuta el agente y almacena el resultado; el cliente hace polling al endpoint de estado (`GET /tasks/{job_id}`) o recibe una notificación por webhook cuando la tarea completa.

La mayoría de los sistemas de producción reales usan un **modelo híbrido**: stateless con persistencia externa para el estado de sesión, y la elección entre síncrono o asíncrono según el perfil de latencia de la tarea. Un sistema de customer support con agentes que completan tareas en 10-30 segundos usará síncrono con streaming; un sistema de análisis de documentos con tareas de 2-10 minutos usará asíncrono con job queue; un sistema de generación de reports nocturnos con tareas de 30-120 minutos usará long-running jobs con checkpointing.

Aspectos técnicos

- **Stateless + external state**: el agente recibe el estado completo en cada request desde Redis (sesión activa) o PostgreSQL (historial persistente); permite escalado horizontal sin sticky sessions; el tradeoff es latencia adicional por serialización/deserialización de estado en cada request
- **Stateful (no recomendado para escala)**: estado en memoria de la instancia; requiere sticky routing en el load balancer; complica rolling deployments y recuperación ante fallos; adecuado solo para prototipos o sistemas de un solo usuario
- **Síncrono con streaming via SSE**: el servidor envía eventos mientras el agente ejecuta; el cliente recibe pasos intermedios en tiempo real; mejora la UX en tareas de 10-60 segundos donde el usuario espera una respuesta; implementado con FastAPI `EventSourceResponse` o equivalente

- **Asíncrono con job queue:** el request encola la tarea en Redis/SQS con un `task_id`; un worker consume y procesa; el cliente hace polling a `GET /tasks/{task_id}` o recibe webhook; obligatorio para tareas de más de 60 segundos
- **Timeout boundaries en ambos modos:** en modo síncrono, definir timeout del servidor (120s para FastAPI/uvicorn) y timeout del agente (`max_steps × estimated_step_latency`); en modo asíncrono, timeout del worker y del total de la tarea (p.ej., 10 minutos máximo por tarea)

Nota del Arquitecto: La decisión entre síncrono y asíncrono no debe tomarse basándose en la latencia promedio de la tarea sino en la latencia del percentil 95. Si el P50 es 20 segundos pero el P95 es 90 segundos, el modo síncrono producirá timeouts del servidor para el 5% de los requests —exactamente los casos más complejos, que son también los más importantes para el usuario. El modo asíncrono con notificación por webhook o polling es más robusto cuando la distribución de latencias tiene una cola larga.

Principio rector

La arquitectura de despliegue de un agente se elige en función del perfil de latencia de la tarea: menos de 30 segundos es candidata a síncrono con streaming; 30 segundos a 10 minutos requiere asíncrono con job queue; más de 10 minutos requiere arquitecturas de long-running jobs con checkpointing en cada paso. Comenzar con el modo incorrecto y migrar en producción tiene un costo operativo alto; la decisión correcta se toma antes del primer despliegue.

Módulo 7 – Capítulo 09 – Sección 02

Gestión del estado entre sesiones: persistencia y recuperación de contexto

Un agente en producción puede ser interrumpido en cualquier punto de su ejecución por razones que no tienen nada que ver con el agente en sí: el servidor puede ser reemplazado durante un rolling deployment mientras la tarea está en curso; el proceso puede terminar inesperadamente por agotamiento de memoria; el usuario puede cerrar la conexión en el paso 15 de 20, con intención de retomar la tarea más tarde; un operador puede pausar la ejecución para insertar una revisión humana. En todos estos casos, la pregunta crítica es: ¿el sistema puede reanudar desde donde quedó, o debe reiniciar desde cero? Para tareas cortas de segundos, el reinicio es aceptable. Para tareas de varios minutos o decenas de pasos, el reinicio completo ante cualquier interrupción no es una propiedad aceptable de un sistema de producción: desperdicia recursos, degrada la experiencia del usuario, y en algunos casos —tareas con efectos secundarios irreversibles ya ejecutados— puede ser directamente incorrecto reejecutar desde el principio.

El **checkpointing** resuelve este problema al serializar periódicamente el estado completo del agente a almacenamiento durable, de forma que una ejecución interrumpida pueda reanudarse desde el último checkpoint guardado en lugar de desde el inicio. En LangGraph, el mecanismo de checkpointing se configura al compilar el `StateGraph`: `graph = workflow.compile(checkpointer=PostgresSaver.from_conn_string(db_url))`. Con este `checkpointer` configurado, LangGraph serializa automáticamente el estado completo del grafo —el `AgentState` con todas sus variables, el historial de mensajes, los nodos ya ejecutados— a PostgreSQL después de completar cada nodo. Si la ejecución se interrumpe en cualquier punto, reanudar es tan simple como llamar `graph.invoke(None, config={"configurable": {"thread_id": "session-abc-123"}})`: LangGraph recupera el último estado guardado para ese `thread_id` y continúa desde el nodo siguiente al último completado, sin re-ejecutar los nodos ya completados.

El **granulo del checkpointing** determina cuánto trabajo puede perderse en el peor caso: con checkpointing por nodo —el comportamiento default de LangGraph— el agente nunca

pierde más de un nodo de trabajo ante una interrupción, porque el estado se persiste después de completar cada nodo. Un granulo más grueso (p.ej., guardar el estado cada 5 nodos) reduce el overhead de escrituras a base de datos pero aumenta el trabajo que puede perderse. Para tareas largas, el checkpointing por nodo es el tradeoff correcto.

El **AgentState serializable** es un requisito de diseño que frecuentemente se ignora hasta que causa problemas. El estado que LangGraph serializa a PostgreSQL debe ser completamente representable en JSON o en tipos nativos de la base de datos: listas, diccionarios, strings, números. Objetos Python que no son serializables —conexiones de base de datos, file handles abiertos, objetos de configuración con estado, referencias a funciones— no pueden incluirse en el **AgentState** del checkpointer. La solución es separar el estado serializable del contexto de ejecución no serializable: el estado incluye los datos del agente (mensajes, observaciones, variables de tarea), mientras que el contexto de ejecución (conexiones de herramientas, credenciales) se reconstruye al reanudar la sesión a partir de la configuración del sistema, no del checkpoint.

El **thread_id como identificador de sesión** es la clave que une cada request de un usuario con su estado persistido. Al reanudar una sesión, el cliente proporciona el **thread_id** de la sesión original; LangGraph recupera el estado asociado a ese ID del checkpointer y continúa la ejecución. Este mecanismo permite múltiples sesiones concurrentes del mismo agente sin interferencia entre ellas: cada sesión tiene su propio **thread_id** y su propio espacio de estado en el checkpointer. La gestión de **thread_id** es responsabilidad del cliente o de la capa de API del sistema: puede usarse el ID de sesión del usuario, un UUID generado al inicio de cada tarea, o el ID de un ticket de soporte si el agente opera en ese contexto.

La **retención y limpieza de checkpoints** es una consideración operativa que los equipos frecuentemente posponen hasta que el almacenamiento crece de forma inesperada. Los checkpoints de sesiones ya completadas no son necesarios para la ejecución futura y pueden purgarse con un TTL corto (7 días). Las sesiones pausadas esperando revisión humana deben retenerse más tiempo (30 días es razonable para la mayoría de los contextos). Las sesiones activas en curso nunca deben purgarse automáticamente. Implementar un job de limpieza periódico que elimine checkpoints según su estado y antigüedad previene el crecimiento indefinido del almacenamiento sin impactar la funcionalidad.

Aspectos técnicos

- **Checkpointing por nodo en LangGraph:**

`compile(checkpointer=PostgresSaver(...))` serializa el `AgentState` completo a PostgreSQL después de cada nodo; `graph.invoke(None, config={"configurable": {"thread_id": "..."}})` reanuda desde el último estado guardado para ese `thread_id`

- **MemorySaver VS SqliteSaver VS PostgresSaver** : `MemorySaver` es en memoria (solo para tests, no sobrevive reinicios), `SqliteSaver` es en archivo local (adecuado para desarrollo y producción de baja escala), `PostgresSaver` es en PostgreSQL (producción de alta escala, soporta múltiples instancias concurrentes)
- **Estado serializable**: el `AgentState` debe contener solo tipos JSON-serializables; objetos no serializables (conexiones DB, file handles) se excluyen del estado y se reconstruyen al reanudar a partir de la configuración del sistema
- **Reanudación parcial correcta**: al reanudar, el agente identifica el último nodo completado via el checkpoint y continúa desde el nodo siguiente; no re-ejecuta nodos ya completados ni repite efectos secundarios (emails enviados, datos modificados) que ya ocurrieron en la ejecución anterior
- **Retención de checkpoints**: TTL diferenciado según estado de la sesión: 7 días para sesiones completadas (no necesarias para reanudación), 30 días para sesiones pausadas esperando revisión humana, indefinido para sesiones activas; job de limpieza periódico (cron diario) que aplica el TTL

Nota del Arquitecto: El error de diseño más costoso en producción es asumir que el checkpointing es solo para alta disponibilidad y omitirlo en deployments "pequeños". El checkpointing también es necesario para el HITL: cuando el agente hace una pausa para solicitar revisión humana, el estado debe persistirse de forma que la aprobación o rechazo humano ocurra en una request completamente diferente, potencialmente horas después. Sin checkpointing, el HITL es arquitectónicamente imposible de implementar correctamente en producción.

Para recordar

El checkpointing no es un lujo de sistemas de alta disponibilidad; es un requisito de correctitud para cualquier agente que ejecuta tareas largas o que implementa HITL. Una tarea de 20 minutos que falla en el minuto 18 sin checkpointing reinicia completamente; con checkpointing, reanuda desde el paso 17 de 20 y termina en 2 minutos adicionales.

Módulo 7 – Capítulo 09 – Sección 03

Timeouts y cancelación: qué hacer cuando un agente no termina en tiempo

Los agentes en producción pueden entrar en estados de no terminación por razones que van desde fallos técnicos hasta limitaciones inherentes del modelo. Un razonamiento circular puede hacer que el agente repita el mismo ciclo de acciones sin convergir hacia un resultado; una herramienta externa puede colgar indefinidamente en lugar de devolver un error limpio; el problema puede ser intrínsecamente más complejo de lo estimado y requerir más pasos que el límite configurado; o el agente puede entrar en un bucle de auto-corrección donde cada intento de resolver un error introduce un error diferente. Sin mecanismos de timeout y cancelación explícitos, estos escenarios producen requests que consumen recursos indefinidamente —créditos de API del LLM, créditos de herramientas externas, CPU y memoria del servidor— sin ninguna garantía de que eventualmente terminarán.

La gestión de timeouts en agentes debe operar en tres capas independientes y complementarias. La primera capa es el **timeout por herramienta individual**: cada invocación de herramienta tiene un tiempo máximo de espera propio, proporcional a la latencia esperada de esa herramienta específica. Una búsqueda web debería terminar en menos de 5-15 segundos; la ejecución de código Python en un sandbox puede tomar hasta 30-60 segundos; una llamada a una API de terceros raramente justifica esperar más de 30 segundos. En Python asíncrono, esto se implementa con `asyncio.wait_for(tool_call, timeout=30)`, que lanza `asyncio.TimeoutError` cuando se supera el límite. El manejo explícito de este error —con un mensaje de observación al agente que describe qué herramienta timeoutó y sugiere un plan alternativo— es más robusto que dejar que el timeout se propague como excepción no manejada.

La segunda capa es el **timeout de sesión total**: el agente como unidad completa tiene un tiempo máximo de ejecución independiente de cuántas iteraciones ejecute. Este límite protege contra el caso donde el agente completa iteraciones rápidamente pero en un bucle infinito: `max_steps=15` previene más de 15 iteraciones, pero si cada iteración tarda 30 segundos en media, 15 iteraciones producen 7.5 minutos de ejecución — que puede ser aceptable o

inaceptable según el SLO del sistema. El timeout de sesión se implementa a nivel del worker asíncrono o del proceso del agente: cuando se supera, el agente debe persistir su estado actual en el checkpointer y devolver el mejor resultado parcial disponible con una nota explicativa de que la tarea no se completó por timeout.

El `max_steps` como límite de iteraciones es la forma más directa de prevenir bucles infinitos en el ciclo de razonamiento-acción. El valor correcto de `max_steps` depende del dominio: para tareas simples de 3-5 pasos, `max_steps=10` proporciona un margen generoso sin riesgo de ejecuciones largas; para tareas de análisis complejas de 10-20 pasos, `max_steps=30` es razonable; para tareas de ingeniería de software que pueden requerir muchas iteraciones de escritura-test-corrección, `max_steps=50` puede ser apropiado. El diseño correcto cuando se alcanza `max_steps` no es lanzar una excepción sino devolver gracefully el resultado parcial con una nota de que la tarea alcanzó el límite de iteraciones, qué fue completado, y qué quedó pendiente. Esta información es valiosa para el usuario y para el equipo que analiza los fallos del sistema.

La **cancelación explícita** es el mecanismo que permite al usuario o al operador del sistema terminar una tarea en ejecución antes de que alcance su límite natural. La cancelación se implementa mediante un **cancellation token** —una referencia compartida al estado de cancelación— que el agente verifica en puntos de control seguros entre iteraciones. La verificación entre iteraciones, no a mitad de una operación individual de herramienta, es crítica: cancelar mientras el agente está a mitad de una escritura en base de datos puede dejar el sistema en un estado inconsistente. La cancelación debe ser limpia: guardar el estado actual en el checkpointer, terminar las operaciones en curso que sean seguras de completar, y devolver una respuesta que confirme que la cancelación fue procesada correctamente.

La **degradación graceful en timeout** —en lugar de fallar con un error genérico— es el estándar de calidad para el comportamiento de un agente de producción ante el límite de tiempo. El agente que completa 8 de 12 subtareas de un análisis antes del timeout debe devolver los resultados de las 8 subtareas completadas, indicar claramente cuáles 4 quedaron pendientes, y proporcionar información suficiente para que el usuario pueda re-invocar la tarea con el trabajo ya hecho como contexto. Esta respuesta es fundamentalmente más útil que un mensaje de error genérico que descarta todo el trabajo completado.

Aspectos técnicos

- **Timeout por herramienta:** `asyncio.wait_for(tool_coroutine, timeout=N)` con manejo explícito de `asyncio.TimeoutError`; el timeout es proporcional a la latencia esperada de la herramienta específica (búsqueda web: 5-15s, ejecución de código: 30-60s, API externa: 5-30s); una observación descriptiva del timeout se devuelve al agente para que pueda razonar sobre alternativas
- **Timeout de sesión total:** timer global que cancela el task asyncio completo cuando se supera el tiempo máximo de sesión; el estado se persiste en el checkpointer antes de retornar; la respuesta incluye el resultado parcial y descripción de qué quedó pendiente
- **`max_steps` como límite de iteraciones:** límite explícito en el número de iteraciones del ciclo agéntico (10-50 según complejidad esperada de la tarea); cuando se alcanza, el agente devuelve el mejor resultado parcial disponible con nota explicativa, no lanza una excepción sin contexto
- **Cancellation token:** referencia compartida al estado de cancelación que el agente verifica entre iteraciones (no a mitad de operaciones individuales); al detectar cancelación, el agente termina limpiamente: guarda estado, completa operaciones en curso seguras, devuelve respuesta de confirmación de cancelación
- **Graceful degradation:** el comportamiento correcto ante timeout o `max_steps` es devolver los resultados parciales disponibles con descripción explícita de qué fue completado y qué quedó pendiente; nunca silenciosamente descartar el trabajo ya realizado

Principio rector

Los timeouts no son configuraciones de performance; son garantías de comportamiento que hacen posible ofrecer SLOs sobre latencia y costo. Un sistema sin timeouts explícitos no puede garantizar cuándo terminará una tarea ni cuánto costará, lo que hace imposible operar el sistema dentro de límites presupuestarios y de calidad de servicio predecibles.

Módulo 7 – Capítulo 09 – Sección 04

Observabilidad agéntica y gestión de presupuesto de tokens

La observabilidad de sistemas tradicionales captura inputs y outputs de los servicios: request recibido, response enviado, latencia, código de estado. Esta granularidad es insuficiente para diagnosticar comportamiento agéntico en producción. Un agente que devuelve una respuesta incorrecta puede haberlo hecho por razones completamente diferentes: el razonamiento del LLM se desvió en el paso 7 de 12; la herramienta de búsqueda devolvió resultados desactualizados; la observación fue mal interpretada en el paso 9; el agente no incorporó una corrección de error que recibió en el paso 11. Sin trazas granulares de cada paso del ciclo de razonamiento-acción, reproducir y diagnosticar un fallo de producción requiere reconstruir manualmente una cadena de eventos que puede tener decenas de pasos, con el agravante de que el entorno puede haber cambiado desde que ocurrió el fallo.

La observabilidad agéntica requiere capturar el ciclo completo con **trazas anidadas** (spans) que representan la estructura jerárquica de la ejecución: la traza raíz corresponde a la sesión completa del agente, con spans hijos para cada iteración del ciclo, y spans nietos para cada llamada al LLM y cada invocación de herramienta dentro de cada iteración. Esta estructura anidada permite navegar la ejecución en dos formas: top-down (¿qué ocurrió en la sesión completa?) y bottom-up (¿en qué contexto específico ocurrió esta invocación de herramienta que falló?). El formato OpenTelemetry (OTLP) está siendo adoptado como estándar para trazas de LLMs, con soporte nativo en Langfuse, Arize Phoenix, y otras plataformas de observabilidad especializadas.

Las **métricas por paso** que deben capturarse en cada span son: la latencia del LLM (tiempo de espera de la respuesta de la API, incluyendo time-to-first-token para streaming), la latencia de la herramienta (tiempo de ejecución de la herramienta, separado de la latencia del LLM), los tokens de input y output del LLM en ese paso, el nombre de la herramienta invocada y sus argumentos, la longitud de la observación retornada, y el costo estimado en USD del paso individual. Estos datos, agregados por sesión y por período de tiempo, producen los dashboards que permiten al equipo entender el comportamiento del agente a escala: cuál es la

latencia típica de cada tipo de tarea, cuáles herramientas son las más lentas, en qué tipos de tareas el agente usa más iteraciones de las esperadas.

LangSmith (LangChain), **Langfuse** (open-source, auto-hosteable), y **Arize Phoenix** son las plataformas de observabilidad más adoptadas en el ecosistema agéntico. Langfuse se destaca por ser open-source y auto-hosteable, lo que lo hace apropiado para contextos con restricciones de privacidad de datos; su integración con LangGraph es sencilla mediante callbacks. LangSmith ofrece la integración más profunda con el ecosistema LangChain, incluyendo herramientas de evaluación y datasets de prueba. Arize Phoenix es especialmente fuerte en evaluación basada en LLM-judge integrada en el pipeline de observabilidad. La elección entre plataformas depende de las restricciones de privacidad, el stack tecnológico existente, y los requerimientos de evaluación integrada.

Gestión de presupuesto de tokens y control de costos

La observabilidad de costos de tokens merece tratamiento propio porque es la dimensión que más frecuentemente sorprende a los equipos que despliegan agentes en producción. Un agente que funciona con costo aceptable en tests puede generar costos inesperadamente altos en producción por razones que no son visibles sin monitoreo específico: el historial de mensajes crece en cada iteración hasta llenar el contexto y producir prompts enormes; las observaciones de herramientas son más largas de lo esperado con datos reales; el agente usa más iteraciones de las esperadas en ciertos tipos de tareas que son más frecuentes en producción que en el test set.

La **fórmula de estimación de costo por tarea** proporciona una referencia de planificación:

```
costo_estimado_por_tarea = pasos_estimados * tokens_por_paso * precio_por_millón_de_tok
```

Para un agente con `pasos_estimados = 10`, `tokens_por_paso = 3.000` (sumando input + output + observaciones de herramientas), y usando Claude 3.5 Sonnet a `$3.00 por millón de tokens de input y $15.00 por millón de output`:

```
tokens_input_por_tarea ≈ 10 × 2.500 = 25.000 tokens → $0.075
tokens_output_por_tarea ≈ 10 × 500 = 5.000 tokens → $0.075
costo_estimado_total ≈ $0.15 por tarea
```

Si el sistema procesa 10.000 tareas/día, el costo estimado es \$1.500/día o \$45.000/mes. Esta estimación es el punto de partida para el dimensionamiento presupuestario; el monitoreo de producción revelará si los valores reales se desvían de la estimación y en qué dirección.

El **token budget middleware** es el patrón de implementación que previene que tareas individuales consuman un número irrazonablemente alto de tokens. Implementado como una función que se ejecuta antes de cada llamada al LLM, verifica el número de tokens del contexto actual y actúa cuando supera un umbral: si el contexto supera el 70% del límite configurado, comprimir el historial (resumir los mensajes más antiguos); si supera el 90%, terminar la tarea con el resultado parcial disponible y una nota de que el contexto fue agotado; si supera el límite de la API del modelo, la llamada fallará con un error —el peor outcome posible.

```
MAX_CONTEXT_TOKENS = 100_000 # presupuesto de tokens por sesión

def check_token_budget(messages: list[dict]) -> tuple[bool, int]:
    """Verifica si el contexto actual excede el presupuesto de tokens."""
    estimated_tokens = sum(len(m["content"].split()) * 1.3 for m in messages)
    if estimated_tokens > MAX_CONTEXT_TOKENS * 0.9:
        return False, estimated_tokens # presupuesto agotado
    return True, estimated_tokens

# En el ciclo del agente, antes de cada llamada al LLM:
budget_ok, current_tokens = check_token_budget(state["messages"])
if not budget_ok:
    # resumir historial o terminar con resultado parcial
    state["messages"] = summarize_history(state["messages"])
```

Las **alertas de anomalías de costo** son la segunda capa de control. Configurar alertas cuando: el costo de una sesión individual supera un umbral (p.ej., \$5 por sesión), el costo promedio por tarea de un período supera el 150% del baseline histórico, o el número de tokens por sesión del percentil 95 supera el doble del P50 (indica outliers que consumen recursos desproporcionados). Estas alertas permiten detectar cambios en el comportamiento del agente —causados por cambios en la distribución de inputs o por modificaciones no previstas de prompts o herramientas— antes de que produzcan impacto presupuestario significativo.

Aspectos técnicos

- **Trazas anidadas (spans):** traza raíz por sesión completa del agente, con spans hijos por iteración y spans nietos por llamada al LLM y por invocación de herramienta; formato OpenTelemetry (OTLP) como estándar emergente; soporte en Langfuse, Arize Phoenix, LangSmith
- **Métricas por paso:** latencia del LLM, latencia de la herramienta, tokens de input/output por paso, costo estimado por paso, nombre y argumentos de la herramienta invocada, longitud de la observación; base para dashboards de comportamiento a escala
- **Session replay:** reproducción paso a paso de la ejecución de un agente usando trazas almacenadas; disponible en LangSmith y Langfuse; esencial para investigar incidentes reportados por usuarios sin necesidad de reproducir el entorno
- **Fórmula de costo por tarea:** $\text{pasos_estimados} \times \text{tokens_por_paso} \times \text{precio} / 1\text{M}$; usar para planificación presupuestaria; validar contra costo real en producción y ajustar si la desviación supera el 50%
- **Token budget middleware:** verificación del número de tokens del contexto antes de cada llamada al LLM; umbral de compresión al 70% (resumir historial), umbral de terminación al 90% (devolver resultado parcial); previene llamadas que superan el límite de la API del modelo
- **Sampling adaptativo de trazas:** en producción de alto volumen, capturar 100% de las sesiones que fallan o superan umbrales de latencia/costo, y un porcentaje configurable (10-20%) de las sesiones exitosas; reduce el costo de almacenamiento sin perder cobertura de los casos críticos

Nota del Arquitecto: El error más frecuente en observabilidad agéntica no es no tener las herramientas correctas sino no configurar alertas sobre las métricas correctas desde el primer día en producción. Un dashboard sin alertas es un instrumento de diagnóstico reactivo; las alertas son el instrumento de detección proactiva. Configurar al menos estas tres alertas desde el despliegue inicial: tasa de sesiones que alcanzan `max_steps` (indica tareas más complejas de lo diseñado), costo medio por sesión (detecta degradaciones de eficiencia), y tasa de fallos de herramientas (detecta cambios en APIs externas antes de que los usuarios lo reporten).

Para recordar

La observabilidad agéntica no es logging de texto plano; es la captura estructurada de cada decisión del ciclo de razonamiento-acción, con la granularidad suficiente para diagnosticar exactamente en qué paso el agente tomó la decisión incorrecta. La gestión de presupuesto de tokens es parte de la observabilidad, no un añadido: un agente que no controla su consumo de tokens no puede ofrecer garantías de costo predecibles en producción.

Módulo 7 – Capítulo 09 – Sección 05

Escalabilidad horizontal de agentes: workers, colas y balanceo de carga

Un agente en producción de alto volumen es, desde el punto de vista de la infraestructura, un consumidor de recursos intensivo y de duración variable: cada tarea puede requerir múltiples llamadas a APIs externas con latencias de cientos de milisegundos cada una, ejecución de código en sandboxes, y ciclos de razonamiento del LLM que pueden tomar segundos por iteración, con una duración total por tarea que va de segundos a minutos. Estas características hacen que una arquitectura de servidor web convencional —donde cada request HTTP ocupa un hilo o coroutine durante la duración de la respuesta— sea ineficiente para escalar agentes: un servidor HTTP estándar con 20 workers concurrentes procesando tareas de 30 segundos cada una tiene un throughput máximo de 40 tareas por minuto, que es insuficiente para muchos sistemas de producción.

El patrón de escalabilidad correcto para agentes es el **sistema de colas con worker pools**. El flujo tiene tres componentes independientes: los **productores** (los clientes que envían tareas, típicamente instancias del API server), la **cola de mensajes** (que actúa como buffer elástico entre productores y workers, implementada con Redis, RabbitMQ, Amazon SQS o Apache Kafka), y los **workers** (instancias dedicadas del agente que consumen tareas de la cola y las ejecutan hasta completarlas). Este desacoplamiento entre la recepción de requests y su procesamiento tiene tres propiedades operativas fundamentales: los productores pueden aceptar tasks nuevas sin importar si hay workers disponibles en ese momento (resiliencia ante picos de demanda), el número de workers puede ajustarse independientemente del número de instancias del API server (escalabilidad diferenciada por capa), y un worker que falla durante el procesamiento devuelve la tarea a la cola para que otro worker la procese (resiliencia ante fallos de instancias).

Celery con Redis o RabbitMQ es el stack de worker queue más establecido en el ecosistema Python para este patrón. Un worker Celery para agentes puede configurarse para ejecutar múltiples agentes concurrentemente usando `asyncio` (apropiado para agentes I/O-bound, que pasan la mayor parte del tiempo esperando respuestas de APIs): `celery -A`

`agent_tasks worker --concurrency=10 --pool=gevent` lanza un worker que procesa hasta 10 tareas concurrentemente usando greenlets, lo que es mucho más eficiente que 10 procesos Python separados para workloads I/O-bound. Para workloads CPU-intensivos (ejecución de código, procesamiento de imágenes), el modelo de procesos separados es más apropiado.

El **auto-scaling basado en métricas de cola** automatiza el ajuste del número de workers en función de la carga real. La señal más directa es la **profundidad de la cola** (número de tareas pendientes de procesamiento): cuando supera un umbral (p.ej., 50 tareas pendientes), el sistema de orquestación añade workers; cuando cae por debajo de otro umbral (p.ej., 5 tareas pendientes), reduce workers para minimizar el costo de infraestructura en períodos de baja demanda. En Kubernetes, este patrón se implementa con **KEDA** (Kubernetes Event-driven Autoscaling), que escala el número de réplicas de los Deployments de workers en función de métricas de colas externas (Redis, SQS, RabbitMQ) sin requerir programación adicional. La combinación de Kubernetes + KEDA + Celery o Dramatiq es un stack de producción probado para escalabilidad horizontal de agentes.

La **dead letter queue (DLQ)** es el mecanismo que previene que tareas irrecuperables bloqueen la cola principal. Cuando una tarea falla repetidamente (supera `max_retries`, típicamente 3), se mueve automáticamente a la DLQ. Allí, un operador humano puede revisar la tarea, diagnosticar la causa del fallo, y decidir si reintentar manualmente (si fue un fallo transitorio), descartar la tarea (si el input es inválido), o crear un ticket de bug (si es un fallo del agente que requiere corrección de código). La DLQ es esencial para la observabilidad de fallos sistémicos: si 50 tareas se acumulan en la DLQ en un período de 1 hora con el mismo mensaje de error, eso es una alerta de que hay un problema sistemático que requiere atención inmediata.

La **priorización de tareas** es una consideración de diseño que frecuentemente se omite hasta que hay un incidente. En un sistema con un único nivel de prioridad, una carga de trabajo batch —análisis nocturnos, generación de reportes periódicos— puede llenar la cola y producir latencias inaceptables para las tareas interactivas de usuario que llegan concurrentemente. Implementar dos colas separadas —una de alta prioridad para tareas interactivas, una de baja prioridad para tareas batch— con workers que priorizan la cola de alta prioridad, previene este problema. En Redis Sorted Sets, la prioridad puede representarse como el score del sorted set: las tareas con menor timestamp de encolamiento tienen mayor prioridad, con la opción de asignar scores artificialmente bajos a tareas marcadas como prioritarias por el sistema.

Aspectos técnicos

- **Queue-based worker pool:** productores (API server) encolan tareas en Redis/SQS con `task_id`, input, configuración; workers consumen y procesan tareas hasta completarlas; desacoplamiento elástico entre recepción y procesamiento
- **Worker concurrency con asyncio:** para agentes I/O-bound, Celery con pool gevent o asyncio permite concurrencia de 5-20 agentes por worker sin overhead de múltiples procesos; para agentes CPU-bound (ejecución de código), pool de procesos separados
- **Auto-scaling con KEDA:** escala el número de réplicas de workers en Kubernetes basándose en métricas de cola (profundidad de cola en Redis/SQS); escala hacia arriba cuando la cola crece, hacia abajo cuando se vacía; reduce el costo de infraestructura en períodos de baja demanda
- **Dead letter queue (DLQ):** tareas que superan `max_retries` se mueven a DLQ para revisión manual; acumulación anómala en DLQ es una señal de alerta de problema sistémico; revisar DLQ periódicamente como parte del proceso de operaciones
- **Priorización de tareas:** dos colas separadas (alta/baja prioridad) con workers que priorizan la cola de alta prioridad; previene que workloads batch produzcan latencias inaceptables para tareas interactivas

Nota del Arquitecto: La métrica de monitoreo más valiosa para el sistema de colas de un agente no es el throughput (tareas procesadas por minuto) sino la **latencia de queue wait** (tiempo desde que una tarea es encolada hasta que un worker comienza a procesarla). Un throughput alto con latencia de queue wait creciente indica que los workers existentes no pueden absorber la demanda actual y que es necesario escalar antes de que los usuarios experimenten degradación. Esta métrica debe estar en el dashboard principal del equipo de operaciones, con una alerta cuando supera el P95 histórico.

Buena práctica

Monitorear la profundidad de cola y la latencia de queue wait como SLIs primarios del sistema de agentes. Un aumento sostenido en la profundidad de cola sin aumento proporcional de workers es la señal más temprana de degradación de capacidad, y actuar sobre esta señal preventivamente —antes de que los usuarios experimenten latencias inaceptables— es la diferencia entre un sistema de producción robusto y uno reactivo.

Módulo 7 – Capítulo 09 – Sección 06

Cierre: un agente en producción es un servicio con requisitos operativos propios

La distancia entre un agente que funciona en el entorno de desarrollo de un ingeniero y un agente que opera de forma confiable bajo carga de producción no es una cuestión de código del agente sino de infraestructura que rodea al agente. El código que implementa el ciclo de razonamiento-acción, la lógica de selección de herramientas, y la gestión del contexto puede ser idéntico en ambos entornos. Lo que separa el prototipo del sistema de producción es todo lo demás: la arquitectura de despliegue que determina cómo el agente escala horizontalmente; el checkpointing que garantiza que las tareas largas pueden reanudarse ante interrupciones; los timeouts y límites de iteraciones que previenen que el agente consuma recursos indefinidamente; la observabilidad de trazas que permite diagnosticar fallos en producción; la gestión de presupuesto de tokens que controla el costo de operación; y el sistema de colas y workers que absorbe los picos de demanda sin degradar la experiencia del usuario.

El error de diseño más costoso en el despliegue de agentes es tratar la infraestructura como un afterthought: "primero lo hacemos funcionar, luego vemos la infraestructura de producción". En la práctica, este enfoque produce sistemas que funcionan en demos con un usuario y un caso de uso perfectamente preparado, pero fallan ante la variabilidad de los inputs reales, la concurrencia de múltiples usuarios simultáneos, los fallos inevitables de herramientas externas, y los cambios de distribución de la demanda a lo largo del tiempo. Diseñar la infraestructura de producción en paralelo con el código del agente —no después— es la única forma de detectar tempranamente las limitaciones arquitectónicas que son costosas de resolver retroactivamente.

Un agente de producción exitoso también es un agente observable: el equipo puede responder las preguntas operativas críticas en cualquier momento utilizando las trazas disponibles, sin necesidad de reproducir el problema manualmente. ¿Cuántas tareas completaron exitosamente en la última semana? ¿Cuál es el costo medio por tarea y cómo ha evolucionado en el último mes? ¿Qué porcentaje de tareas alcanzan `max_steps`? ¿Cuáles son las herramientas con mayor tasa de fallos? ¿Qué tipos de tareas tienen la latencia más alta? La

capacidad de responder estas preguntas con datos —no con suposiciones— es el indicador más directo de madurez operativa de un sistema agéntico.

La conexión con el capítulo siguiente —evaluación y métricas— es directa y relevante: los datos que el sistema de observabilidad del Capítulo 09 captura son la materia prima de la evaluación continua que el Capítulo 10 examina. Las trazas de sesiones en producción alimentan los datasets de evaluación; las métricas de costo y latencia informan las decisiones de qué modelos y configuraciones son viables para el dominio; los patrones de fallos detectados en las trazas guían los ciclos de mejora del agente. Observabilidad y evaluación no son actividades separadas sino dos partes del mismo ciclo de mejora continua.

Para recordar

Un agente que funciona en el laptop del desarrollador pero no en producción no es un agente; es un prototipo. La diferencia la pone la infraestructura de despliegue, no el código del agente: stateless con persistencia externa, checkpointing por nodo, timeouts explícitos en todas las capas, observabilidad de trazas con alertas, gestión de presupuesto de tokens, y escalabilidad horizontal con auto-scaling basado en métricas de cola.

"The ability to deploy software reliably, at any time, is fundamental to building products that people trust." — Jez Humble y David Farley, "Continuous Delivery" (2010); en sistemas agénticos, "software" incluye el LLM, los prompts, las herramientas, el grafo de estado, y la configuración de despliegue — cualquiera de estos componentes puede cambiar en un deployment y debe poder desplegarse, monitorearse y revertirse con el mismo rigor que cualquier cambio de código.